

Lecture 8: Applicatives

COMP2221: Functional programming

Maximilien Gadouleau

`m.r.gadouleau@durham.ac.uk`

- Functors abstract the idea of mapping a function over each element of a structure
- We now generalise to allow functions with any number of arguments to be mapped
- More precisely, we'd like to implement a hierarchy of `fmap` functions as such:

```
fmap0 :: a -> f a
fmap1 :: (a -> b) -> f a -> f b
fmap2 :: (a -> b -> c) -> f a -> f b -> f c
fmap3 :: (a -> b -> c -> d) -> f a -> f b -> f c -> f d
.
.
.
```

- Using the idea of currying, a version of `fmap` for functions of an arbitrary number of arguments can be constructed in terms of two basic functions of the following types:

```
pure  :: a -> f a
(<*>) :: f (a -> b) -> f a -> f b
```

- `pure` converts a value of type `a` into a structure of type `f a`
- `<*>` is a generalised form of function application, where all inputs and the output are all contained in `f` structures
- A typical use looks like this:

```
pure g <*> x1 <*> x2 <*> ... <*> xn
```

such expressions are said to be in *applicative style*

- So we have our generalised `fmap` functions as follows:

```
fmap0 :: a -> f a
fmap0 = pure
```

```
fmap1 :: (a -> b) -> f a -> f b
fmap1 g x = pure g <*> x
```

```
fmap2 :: (a -> b -> c) -> f a -> f b -> f c
fmap2 g x y = pure g <*> x <*> y
```

```
fmap3 :: (a -> b -> c -> d) -> f a -> f b -> f c -> f d
fmap3 g x y z = pure g <*> x <*> y <*> z
```

```
.
```

```
.
```

```
.
```

Applicative functors

- The class of functors that support `pure` and `<*>` are called *applicatives*

```
class Functor f => Applicative f where
  pure :: a -> f a
  <*> :: f (a -> b) -> f a -> f b
```

- They also need to satisfy the applicative laws:

```
pure id <*> x      = x
pure (g x)         = pure g <*> pure x
x <*> pure y       = pure (\g -> g y) <*> x
x <*> (y <*> z)     = (pure (.) <*> x <*> y) <*> z
```

- Those laws guarantee that any well-typed expression using `pure` and `<*>` can be written in applicative style

- It is straightforward to make the functor type `Maybe` into an applicative:

```
instance Applicative Maybe where
  -- pure :: a -> Maybe a
  pure = Just

  -- (<*>) :: Maybe (a -> b) -> Maybe a -> Maybe b
  Nothing <*> = Nothing
  (Just g) <*> mx = fmap g mx
```

- In this form, the applicative style supports a form of exceptional programming, where we can apply pure functions to arguments that may fail without the need to manage the propagation of failures ourselves

The list applicative

- The list type is an applicative, with the following instance declaration in the standard prelude:

```
instance Applicative [] where
  -- pure :: a -> [a]
  pure x = [x]

  -- (<*>) :: [a -> b] -> [a] -> [b]
  gs <*> xs = [g x | g <- gs, x <- xs]
```

- So for instance,

```
Prelude> pure (+1) <*> [1,2,3]
[2,3,4]
```

```
Prelude> pure (+) <*> [1,2,3] <*> [44,55]
[45,56,46,57,47,58]
```

The IO applicative

- The `IO` type can be made into an applicative as follows:

```
instance Applicative IO where
  -- pure :: a -> IO a
  pure = return

  -- (<*>) :: IO (a -> b) -> IO a -> IO b
  mg <*> mx = do {g <- mg; x <- mx; return (g x)}
```

- For instance, a function that reads a given number of characters from the keyboard can be defined as follows:

```
getChars :: Int -> IO String
getChars 0 = return []
getChars n = pure (:) <*> getChar <*> getChars (n-1)
```

Effectful programming

- The common theme between these instances is that they all deal with *effects*
- In each case, the applicative machinery allows us to write programs in a familiar applicative style (functions applied with arguments – albeit with the use of `<*>`)
- The main difference is that the arguments are no longer plain values but may also have effects, such as the possibility of failure, having many ways to succeed, or performing I/O actions
- Applicative functors provide a uniform approach to *effectful programming*, with the precise form of effects that are permitted is prescribed by the particular applicative